

Chapter 1

Multi-armed Bandits

Contents

1	Introduction	2
2	k -armed Bandit Problem	2
3	Balancing between Exploring and Exploiting	2
3.1	Action-Value Methods	2
3.1.1	Estimation of Value	2
3.1.1.1	Methods for Stationary Bandit Problem	2
3.1.1.2	Methods for Non-stationary Bandit Problem	3
3.1.2	Selection of Actions	4
3.1.2.1	Greedy Action Selection	4
3.1.2.2	ϵ -greedy Selection	4
3.1.2.3	Upper-Confidence-Bound Action Selection	5
3.1.3	Optimistic Initial Values	5
3.2	Gradient Bandit Algorithm	5
3.2.1	Algorithm	6

1 Introduction

In this chapter, we discuss about the evaluative aspect of reinforcement learning in simplified setting. We consider situation with only one *state*, with multiple choices. This can be understood as *k*-armed bandit problem.

2 *k*-armed Bandit Problem

In *k*-armed bandit problems, we can pick one of the *k* actions at time step *t*, and receive the reward and then determine the action of next time step *t* + 1, using the reward. Our objective is to maximize the total expected reward over some time period. Following are some notations and definitions:

- Action selected on time step *t*: A_t ;
- Reward given on time step *t*: R_t ;
- The expected reward of selected action *a* (called value): $q_*(a) = \mathbb{E}[R_t | A_t = a]$

If the values of actions are known exactly, we just have to select the action which has largest value at each time step. So we assume that the exact values of actions are not known, and this is the usual situation, too. Since we do not know exact values, we try to estimate the values.

3 Balancing between Exploring and Exploiting

3.1 Action-Value Methods

Since we assume that the exact value of each actions are not known, we try to estimate the value of actions, and use it to make action selection decisions. This is collectively called as **action-value methods**.

3.1.1 Estimation of Value

3.1.1.1 Methods for Stationary Bandit Problem The reward probabilities does not change over time in *stationary problems*. The following is a natural way of estimating the value of actions.

Definition 3.1.1 (Sample-average Method)

Let A_i and R_i be the action selected at time step *i* and the corresponding reward. Then, the **sample-average** method estimates the value of action *a* at time *t* as following:

$$Q_t(a) = \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}}$$

It is guaranteed to converge to true value, $q_*(a)$, as the time step increases. This can be verified with the law of large numbers.

Considering computational efficiencies, recording all the rewards of each actions selected each time is not memory-efficient. We can use incremental method to implement the sample-average method efficiently.

Algorithm 1: Incremental Sample Average Algorithm

Input : t current time step
 a selected action in time step t
 n the times of action a was selected until time step t (excluding current selection)
 R_n reward given to n -th selected action a
 $Q_t(a)$ estimation of value of action a in current time step t

Output: $Q_{t+1}(a)$ new estimate of value of action a

1 $Q_{t+1}(a) \leftarrow Q_t(a) + \frac{1}{n}(R_n - Q_t(a))$

2 **return** $Q_{t+1}(a)$

3.1.1.2 Methods for Non-stationary Bandit Problem The averaging methods discussed above are not appropriate for *non-stationary* bandit problem, whose reward probabilities changes over time. For non-stationary bandit problem, we give more weight to recent rewards of action than to long-past rewards. We can implement this via using the moving average.

Algorithm 2: Weighted Average Method Algorithm

Input : t current time step
 a selected action in time step t
 n the times of action a was selected until time step t (excluding current selection)
 R_n reward given to n -th selected action a
 $Q_t(a)$ estimation of value of action a in current time step t
 $\alpha \in (0, 1]$ constant step size

Output: $Q_{t+1}(a)$ new estimate of value of action a

1 $Q_{t+1}(a) \leftarrow Q_t(a) + \alpha(R_n - Q_t(a))$

2 **return** $Q_{t+1}(a)$

We can verify that this algorithm weakens the effect of rewards given earlier. We first assume that $Q_n(a)$ is the value of action a at the time step when the action a is selected for n times.

$$\begin{aligned} Q_{n+1}(a) &= Q_n(a) + \alpha(R_n - Q_n(a)) \\ &= \alpha R_n + (1 - \alpha)Q_n(a) \\ &= \alpha R_n + (1 - \alpha)(\alpha R_{n-1} + (1 - \alpha)Q_{n-1}(a)) = \alpha R_n + (1 - \alpha)\alpha R_{n-1} + (1 - \alpha)^2 Q_{n-1}(a) \\ &= \alpha R_n + (1 - \alpha)\alpha R_{n-1} + (1 - \alpha)^2 \alpha R_{n-2} + \cdots + (1 - \alpha)^{n-1} \alpha R_1 + (1 - \alpha)^n Q_1(a) \\ &= (1 - \alpha)^n Q_1(a) + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i \end{aligned}$$

As one can see, the earlier rewards decreases exponentially.

The step size α can vary from step to step. Then, the conditions required for $Q_t(a)$ to converge to true action value with probability 1 is given as:

Theorem 3.1.1 (Conditions for Convergence)

Let $\alpha_n(a)$ be the step-size parameter used to process the reward received after the n -th selection of action a . Then, $Q_t(a)$ converges to $q_*(a)$ as $t \rightarrow \infty$ with probability 1 when following conditions are satisfied:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \text{ and } \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

But these varying step size is seldom used in applications and empirical search, since it often converge slowly or need considerable tuning in order to obtain a satisfactory convergence rate.

3.1.2 Selection of Actions

3.1.2.1 Greedy Action Selection The simplest way of action selection is to select the action with the largest value, which is called the *greedy action*.

Algorithm 3: Greedy Action Selection Algorithm

Input : $\mathcal{A}$ set of all possible actions
 t current time step
 $Q_t(a)$ estimated value of action a at time step t
Output: A_t action selection at time t

```
1  $A_t \leftarrow \arg \max_a Q_t(a)$ 
2 return  $A_t$ 
```

3.1.2.2 ε -greedy Selection Since what we can know is only the estimate of the value in current time step, the greedy action may not be the best action choice in the long run. Selecting the action other than greedy action is called **exploring**. One of the simple method of exploration is to select non-greedy actions with certain probability ε .

Algorithm 4: ε -greedy Selection

Input : $\mathcal{A}$ set of all possible actions
 t time step
 $Q_t(a)$ estimation of value of action a on time step t
 $\varepsilon \in [0, 1]$ probability
Output: A_t action selection at time t

```
1  $p \sim \mathcal{U}(0, 1)$ 
2 if  $p > \varepsilon$  then
3   |  $A_t \leftarrow \arg \max_a Q_t(a)$ 
4 else
5   |  $A_t \sim \mathcal{A}$ 
6 return  $A_t$ 
```

With probability $1 - \varepsilon$, the algorithm exploits. However, with probability ε , the algorithm explores. Note that the exploration does not exclude the greedy action.

3.1.2.3 Upper-Confidence-Bound Action Selection UCB action selection takes into account both how close the estimates of action values are to being maximal and the uncertainties in those estimates.

Algorithm 5: Upper-Confidence-Bound Action Selection

Input : $\mathcal{A}$ set of all possible actions
 t time step
 $Q_t(a)$ estimation of value of action a on time step t
 $N_t(a)$ the number of times action a was selected prior to time t
 c degree of exploration
Output: A_t action selection at time t

```

1  $A_t \leftarrow \arg \max_a \left\{ Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right\}$ 
2 return  $A_t$ 

```

The square root term can be seen as uncertainty term. Each time a is selected, the uncertainty term decreases. On the other hand, when a is not selected for a long time, the uncertainty increases.

The UCB action selection method cannot be extended to general non-stationary bandit problems, and has difficulty dealing with large state spaces.

3.1.3 Optimistic Initial Values

By simply setting the initial values $Q_0(\cdot)$ optimistically, we can encourage exploration easily. First, the algorithm sets initial values equally with high, optimistic values. Then the algorithm selects action, like normal methods. Since the values were highly optimistic, the rewards that the algorithm receives will likely be disappointing, encouraging other actions to be selected next time. The result is that all actions are tried several times before the value estimates converge.

This method can outperform ε -greedy method with realistic initial values when combined with greedy action selection method. However, it cannot be used in non-stationary problems, since its drive for exploration is effective only in the beginning steps.

3.2 Gradient Bandit Algorithm

Instead of estimating the action values and deciding which action to execute depending on the estimates, we can learn the *preference* of actions and use it to maximize the expected reward. This preference has no interpretation in terms of rewards; the higher the preference, the more often the action is selected. The preference is not absolute; it is relative values among the actions. If the preferences of actions are same, for example, 1000, the probability of each action being selected equals each other. We denote the preferences of action a in time step t as $H_t(a)$. The probabilities of choosing the action a at time step t is usually calculated using the *softmax* function, given as:

$$\mathbb{P}\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}}, \quad (1.1)$$

and we denote it as $\pi_t(a)$.

3.2.1 Algorithm

Algorithm 6: Gradient Bandit Algorithm

Input : $\mathcal{A}$ set of all possible actions
 $\{H_0(a)\}_{a \in \mathcal{A}}$ initial preferences
 α step size
Output: $\{H(a)\}$ final preferences

```

1  $t \leftarrow 0$ 
2  $\bar{R}_t \leftarrow 0$ 
3 repeat
4   foreach  $a \in \mathcal{A}$  do
5      $\pi_t(a) \leftarrow \frac{e^{H_t(a)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}}$ 
6    $A_t \sim \text{Categorical}(\{\pi_t(a)\}_{a \in \mathcal{A}})$ 
7    $R_t \leftarrow$  the given reward
8    $\bar{R}_t \leftarrow$  the average (or weighted average for nonstationary) of rewards up through and including time  $t$ 
9   foreach  $a \in \mathcal{A}$  do
10     $H_{t+1}(a) \leftarrow H_t(a) + \alpha(R_t - \bar{R}_t)(\mathbb{1}_{a=A_t} - \pi_t(a))$ 
11   $t \leftarrow t + 1$ 
12 until convergence
13 return  $\{H_t(a)\}_{a \in \mathcal{A}}$ 

```

Remark 3.2.1 (Gradient Bandit Algorithm as Stochastic Gradient Ascent)

The gradient bandit algorithm can be derived using stochastic approximation to gradient ascent. First of all, our objective is to maximize the expected reward:

$$\mathbb{E}[R_t] = \sum_{a \in \mathcal{A}} \pi_t(a) q_*(a) \quad (1.2)$$

where $\pi_t(a)$ is given as 1.1. The update equation of exact gradient ascending would then be

$$H_{t+1}(a) = H_t(a) + \alpha \frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} \quad (1.3)$$

Taking a closer look at the update equation, we can get the following result;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \left\{ \sum_{x \in \mathcal{A}} \pi_t(x) q_*(x) \right\} \quad (1.4)$$

$$= \sum_x q_*(x) \frac{\partial \pi_t(x)}{\partial H_t(a)} \quad (1.5)$$

$$= \sum_x (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} \quad (1.6)$$

where B_t is called *baseline*, and can be any value that does not depend on x . Note that the sum of probability is always 1, and this enables the equality of 1.6. Next we multiply equation by $1 = \frac{\pi_t(x)}{\pi_t(x)}$;

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \sum_x \pi_t(x) (q_*(x) - B_t) \frac{\partial \pi_t(x)}{\partial H_t(a)} / \pi_t(x) \quad (1.7)$$

$$= \mathbb{E} \left[(q_*(A_t) - B_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \quad (1.8)$$

$$= \mathbb{E} \left[(R_t - \bar{R}_t) \frac{\partial \pi_t(A_t)}{\partial H_t(a)} / \pi_t(A_t) \right] \quad (1.9)$$

where we have chosen the baseline $B_t = \bar{R}_t$ and substituted $q_*(A_t) = R_t$. Note that

$$\frac{\partial \pi_t(x)}{\partial H_t(a)} = \frac{\partial}{\partial H_t(a)} \pi_t(x) \quad (1.10)$$

$$= \frac{\partial}{\partial H_t(a)} \left[\frac{e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} \right] \quad (1.11)$$

$$= \frac{\frac{\partial e^{H_t(x)}}{\partial H_t(a)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} \frac{\partial \sum_{b \in \mathcal{A}} e^{H_t(b)}}{\partial H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.12)$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)} \sum_{b \in \mathcal{A}} e^{H_t(b)} - e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.13)$$

$$= \frac{\mathbb{1}_{a=x} e^{H_t(x)}}{\sum_{b \in \mathcal{A}} e^{H_t(b)}} - \frac{e^{H_t(x)} e^{H_t(a)}}{\left(\sum_{b \in \mathcal{A}} e^{H_t(b)} \right)^2} \quad (1.14)$$

$$= \mathbb{1}_{a=x} \pi_t(x) - \pi_t(x) \pi_t(a) \quad (1.15)$$

$$= \pi_t(x) (\mathbb{1}_{a=x} - \pi_t(a)). \quad (1.16)$$

Substituting this result to 1.9, we get

$$\frac{\partial \mathbb{E}[R_t]}{\partial H_t(a)} = \mathbb{E} [(R_t - \bar{R}_t) \pi_t(A_t) (\mathbb{1}_{a=A_t} - \pi_t(a)) / \pi_t(A_t)] \quad (1.17)$$

$$= \mathbb{E} [(R_t - \bar{R}_t) (\mathbb{1}_{a=A_t} - \pi_t(a))]. \quad (1.18)$$

Evaluating this expectation by sampling and substituting to 1.3 leads to

$$H_{t+1}(a) = H_t(a) + \alpha (R_t - \bar{R}_t) (\mathbb{1}_{a=A_t} - \pi_t(a)) \quad (1.19)$$